BTS Services Informatiques aux Organisations
Fiche hebdomadaire de suivi de stage

PARCOURS SISR ☐	**PARCOURS SLAM** ☑

Semaine n° 2 du **12 janvier** au **16 janvier** 2026

Nom de l'étudiant : PORTOLLEAU Anaïs

Nom de l'organisation d'accueil : DELIA TECHNOLOGIES

Nom, adresse e-mail et/ou téléphone du tuteur au sein de l'organisation :

Tuteur : Julien Relandeau
Contacter Pauline Francheteau : 06 14 95 06 49

Nom du tuteur école : Ouafâa Laouija / Nicolas Bourgeois

Rappel du contexte *(activité de l'organisation, nom du service d'accueil et rôle, projet éventuel, …)* :

DELIA TECHNOLOGIES est une ESN (Entreprise de Services du Numérique) fondée en 2019 et basée à Nantes. Elle accompagne ses clients dans la conception, le développement et la maintenance de solutions numériques innovantes, en apportant une expertise technique adaptée à leurs besoins. Une seconde ESN a également été créée en 2021 à Rennes.

À la fin de l'année 2025, une nouvelle version majeure de Node.js ainsi qu'une évolution de l'API Pennylane ont été déployées. Cette évolution technologique a rendu nécessaire la mise à jour de plusieurs projets afin de garantir leur compatibilité, leur sécurité et leur bon fonctionnement dans le cadre de leur exploitation en 2026.

Dans ce contexte, j'ai été amenée à intervenir sur une application existante nommée **Lucas 2 Penny**. Il s'agit d'une application métier permettant notamment la gestion des factures, des données internes de l'entreprise ainsi que d'autres fonctionnalités essentielles. Le projet n'ayant pas été migré depuis une mise à jour datant d'environ juin 2025, une migration vers la nouvelle version était indispensable.

Cette application repose sur l'utilisation de **deux API externes nommées Lucca et Pennylane**.

Mon rôle a consisté à contribuer à cette évolution technique en adaptant le code existant, en identifiant les incompatibilités liées à la montée de la version 2 de l'application, ainsi qu'aux évolutions de l'une des API qui est Pennylane.

Compte rendu du travail effectué *(liste des tâches avec leur description)* :

Suivi et vérifications :

- Vérification du travail réalisé la semaine précédente afin de s'assurer que l'ensemble des fonctionnalités était toujours opérationnel.

- Réunion avec le commercial de l'entreprise pour échanger sur les besoins et la présentation de son métier et le lien avec notre projet

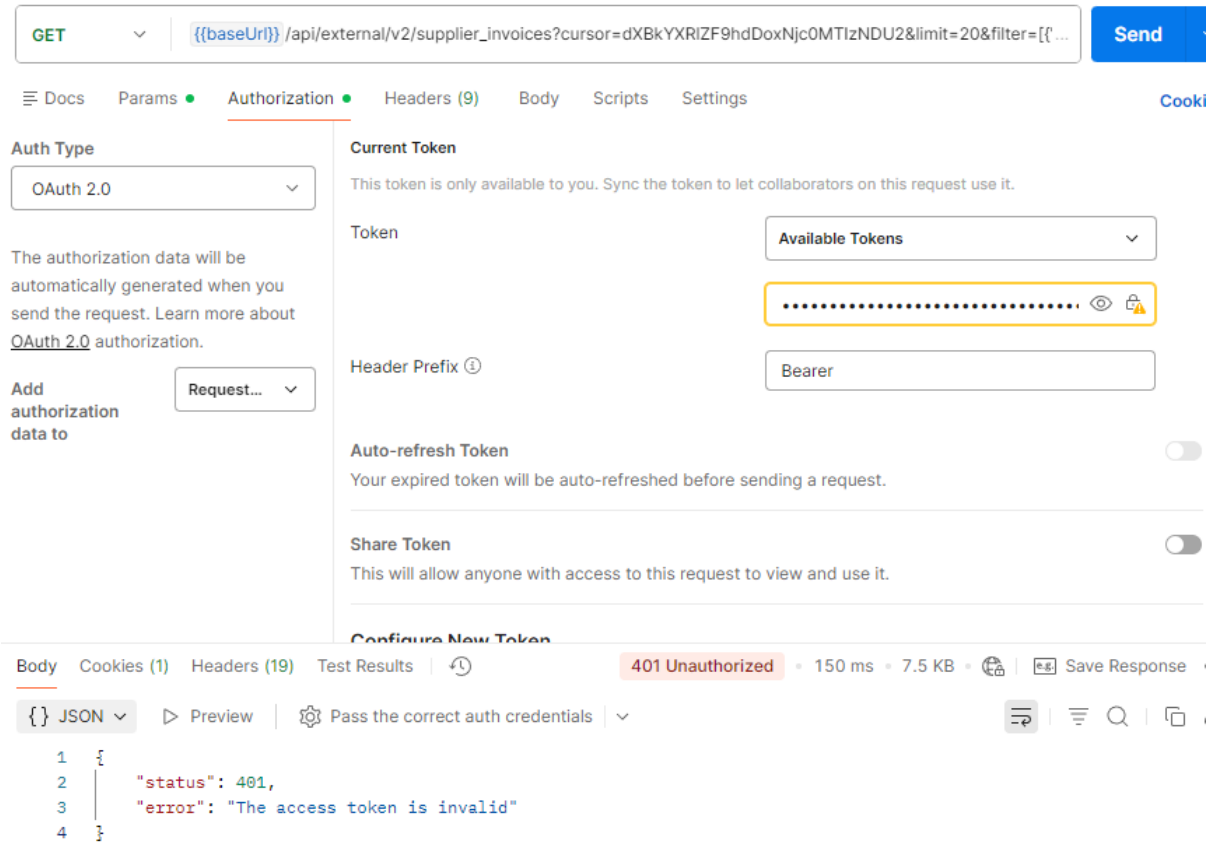
Problèmes techniques rencontrés et résolutions

- Problème identifié dans une fonction utilisant un switch/case, générant l'erreur suivante :
  « La configuration des clés d'API Lucca, Pennylane semble incorrecte ».
  → Suppression de certains cas dans le switch afin de corriger le comportement et rétablir le bon fonctionnement.

- Détection d'un bug dans le fichier supplies_inv_services, qui ne communiquait pas correctement avec l'API.
  → Ce dysfonctionnement était à l'origine des problèmes rencontrés lors de l'affichage de la page contenant les factures et les informations associées.

Problème de configuration des variables d'environnement :

- Analyse des logs indiquant que la constante process.env.PENNY LANE API_KEY était indéfinie.
  → Réalisation de tests avec **Postman** pour vérifier l'hypothèse d'un problème lié au token de l'API.
  → Les tests ont confirmé que le token utilisé était invalide.
  → Nécessité de demander la régénération d'un token API, n'ayant pas les droits nécessaires pour effectuer cette action.



- Utilisation de la console du navigateur pour identifier d'autres erreurs potentielles.

Problèmes d'accès et de permissions :

- Difficultés rencontrées concernant l'accès aux données, dues à une perte d'autorisation pour accéder à l'application (temps important consacré à l'identification du problème).

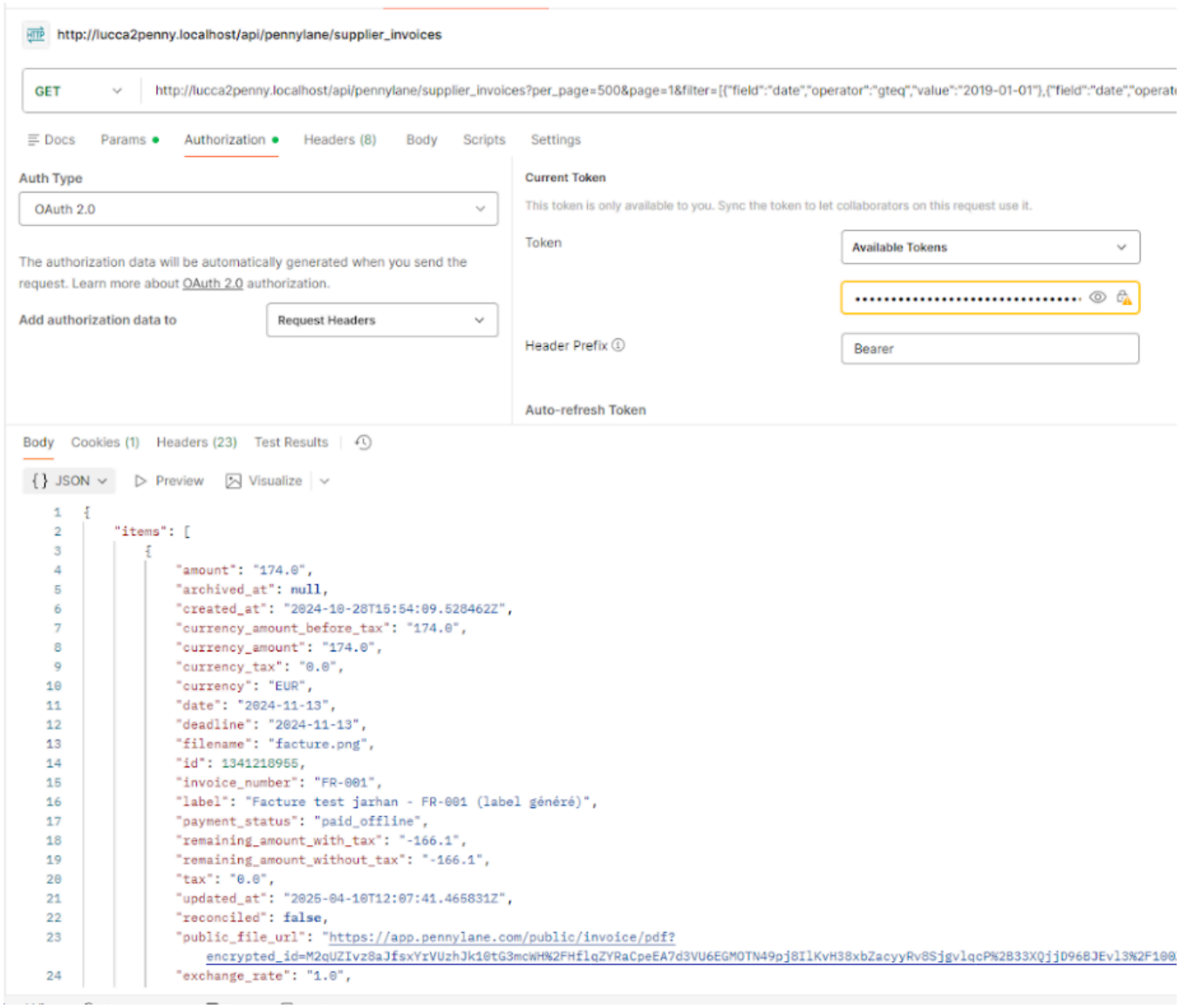
- Utilisation de la console Chrome et de console.log pour analyser les données liées aux déclarations de factures et aux transactions.

```
DevTools - lucca2penny.localhost/declaration-by-transaction
Éléments  Console  Sources  Réseau  Performances  Mémoire  Appli  Confidentialité et sécurité  Lighthouse  Enregistreur
top ▾  Filtrer  Niveaux
Fetching Pennylane supplier invoices: ▸ {month: Wed Jan 14 2026 09:36:33 GMT+0100 (heure normale d'Europe centrale), page: 1, filter: Array(2)}
Pennylane request config: ▸ {url: 'supplier_invoices', method: 'get', headers: AxiosHeaders, params: {…}, baseURL: '/api/pennylane/'}
▸ GET http://lucca2penny.localhost/api/pennylane/supplier_invoices?per_page=500&p…d%22:%22date%22,%22operator%22:%22lteq%22,%22value%22:%222026-01-31%22
▸ Pennylane API error: ▸ {status: 401, statusText: 'Unauthorized', data: {…}, url: 'supplier_invoices', method: 'get'}
▸ ▸ AxiosError {message: 'Request failed with status code 401', name: 'AxiosError', code: 'ERR_BAD_REQUEST', config: {…}, request: XMLHttpRequest, …}
Fetching Pennylane supplier invoices: ▸ {month: Wed Jan 14 2026 09:36:33 GMT+0100 (heure normale d'Europe centrale), page: 1, filter: Array(2)}
Pennylane request config: ▸ {url: 'supplier_invoices', method: 'get', headers: AxiosHeaders, params: {…}, baseURL: '/api/pennylane/'}
▸ GET http://lucca2penny.localhost/api/pennylane/supplier_invoices?per_page=500&p…d%22:%22date%22,%22operator%22:%22lteq%22,%22value%22:%222026-01-31%22
▾ Pennylane API error: ▸ {status: 401, statusText: 'Unauthorized', data: {…}, url: 'supplier_invoices', method: 'get'}
  (anonyme)                    @ api.ts:29
  Promise.then
  getFromPennylaneApi          @ api.ts:46
  fetchPennylaneSupplierInvoices @ supplies_inv.service.ts:53
  (anonyme)                    @ PennylaneGenerateDec…Transactions.tsx:31
  Afficher les frames de la liste des éléments à ignorer
▾ ▸ AxiosError {message: 'Request failed with status code 401', name: 'AxiosError', code: 'ERR_BAD_REQUEST', config: {…}, request: XMLHttpRequest, …}
  onError      @ react-query.js?v=3df8db67:910
  reject2      @ react-query.js?v=3df8db67:493
  (anonyme)    @ react-query.js?v=3df8db67:542
  Promise.catch
  run2         @ react-query.js?v=3df8db67:532
  Retryer2     @ react-query.js?v=3df8db67:560
  fetch        @ react-query.js?v=3df8db67:891
  executeFetch @ react-query.js?v=3df8db67:1947
  fetch        @ react-query.js?v=3df8db67:1940
  refetch      @ react-query.js?v=3df8db67:1924
  onFocus      @ react-query.js?v=3df8db67:759
  (anonyme)    @ react-query.js?v=3df8db67:1133
  (anonyme)    @ react-query.js?v=3df8db67:1132
  batch        @ react-query.js?v=3df8db67:580
```

- Test de l'URL avec la méthode GET et réussite à faire répondre l'API **Lucca**, confirmant son bon fonctionnement avec l'envoie de cet url :

```
http://lucca2penny.localhost/api/pennylane/supplier_invoices?per_page=500&page=1&filter=[{"field":"date","operator":"gteq","value":"
```

qui nous renvoie bien un résultat de facture :

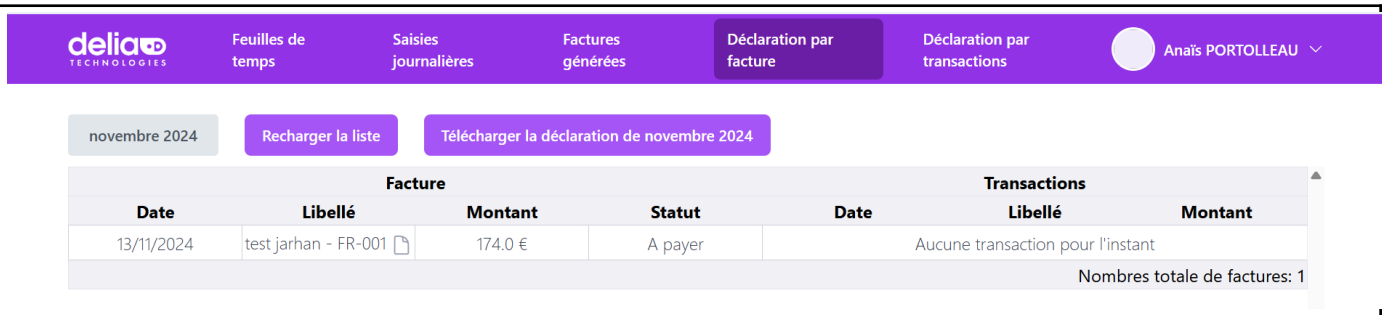


Affichage des factures et corrections associées

- Après résolution des différents bugs, affichage réussi des factures dans un tableau, conformément au fonctionnement attendu (au moins une facture disponible).

- Modifications apportées dans le fichier TableOfInvoicesRow.tsx afin d'adapter la méthode d'affichage et d'utiliser correctement les données de factures (invoices).

- Modification de la méthode getFromPennylaneApi afin de permettre l'affichage des factures récupérées via l'API Pennylane.



Pour pouvoir afficher voici les modifications dans un fichier nommé TableOfInvoicesRow.tsx :

```
27      const transactions: Transaction[] = Array.isArray(matched_transactions)
28          ? matched_transactions
29          : [];
30
31      const firstMatchedTransaction = transactions.slice(0, 1);
32      const othersMatchedTransaction = transactions.slice(1);
33
```

- Modification de la méthode pour utiliser des invoices :

```
    const datas = await getFromPennylaneApi(
        PennylaneApiEndpoint.Supplier,
        cursor,
        filters
    );

    let invoices: IPennylaneSupplierInvoice[] = datas.items ?? [];

    if (datas.has_more && datas.next_cursor) {
        const nextInvoices = await fetchPennylaneSupplierInvoices(
            month,
            sixMonths,
            datas.next_cursor
        );
        invoices = invoices.concat(nextInvoices);
    }

    return invoices;
};
```

- Modification de la méthode getFromPennylanbeApi pour afficher les factures grâce à l'API PennyLane :

```
export const getFromPennylaneApi = (
    endpoint: PennylaneApiEndpoint,
    cursor: string | null = null,
    filters: IPennylaneFilter[] = []
) => {
    const params: Record<string, any> = {
        per_page: perPage,
    };

    if (filters.length > 0) {
        params.filter = JSON.stringify(filters);
    }

    if (cursor) {
        params.cursor = cursor;
    }

    return pennylaneAxiosInstance
        .get(endpoint, { params })
        .then((response) => response.data);
};
```

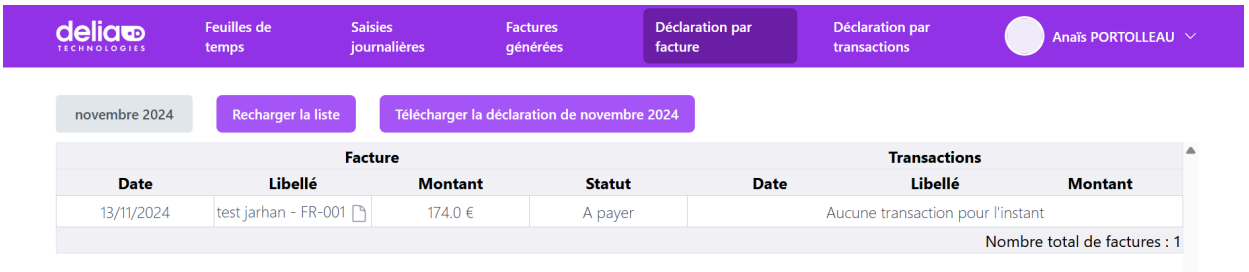
Nettoyage du code

- Nettoyage du code et ajout de documentation du projet côté backend et frontend.

- Suppression d'une méthode devenue inutile dans product.service.ts : fetchPennylaneProductById (GET /products/{id}), la méthode n'était pas utilisée donc plus nécessaire donc je l'ai supprimée pour ne pas alourdir le code.

- Mise en place d'un tableau par défaut afin d'éviter une erreur d'affichage lorsqu'aucun résultat n'est disponible.

- Ajustement de l'affichage lorsqu'une facture existe ainsi qu'un tableau par défaut :



TESTS :

- Vérification des tests unitaires pour vérifier le bon fonctionnement de l'application.

Observations éventuelles :

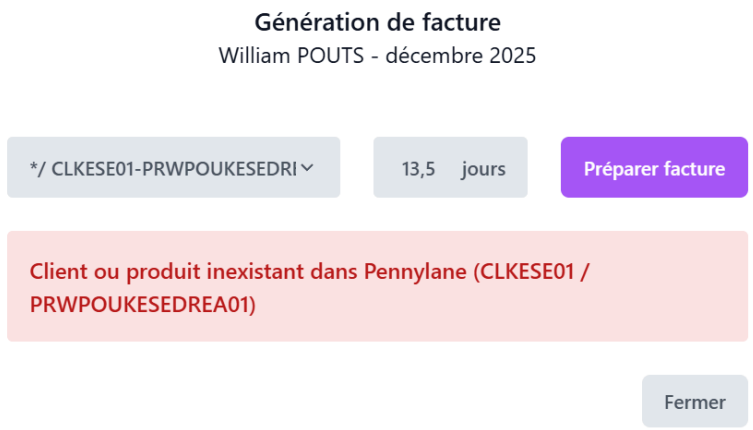
Une partie importante du temps a été consacrée à des tests et analyses via la console Chrome (utilisation de console.log, inspection des requêtes, etc.) afin de tenter de récupérer et comprendre les données liées aux déclarations de factures et aux transactions. J'ai eu énormément de difficultés à afficher les résultats qu'on peut récupérer grâce à l'API dans un tableau.

Beaucoup de problèmes à tester car je n'ai pas les droits de données sur l'API PennyLane qui permet de voir si des factures / transactions existent ou pour en ajouter. Une seule facture de disponible donc pas facile de tester.

Objectifs pour la semaine suivante *(obligatoire)* :

- Vérification des données dans la BDD si elle correspondent bien aux données récupérées dans l'application

- Préparation d'une nouvelle fonctionnalité de génération de factures quand on est essai de générer une facture d'un salarié et qu'on veut préparer la facture cela nous renvoie : "Request failed with status code 400" comme on peut le voir ci-dessous :



→ Pourtant il **existe bien en base** , donc il faudra que j'essaie de régler ce soucis pour pouvoir migrer cette fonctionnalité en V2.